2013 16th Euromicro Conference on Digital System Design

A Distributed BIST Scheme for NoC-based Memory Cores

Bibhas Ghoshal*, Indranil Sengupta†
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Kharagpur, 721302, India
Email:*bibhas@cse.iitkgp.ernet.in,†isg@iitkgp.ac.in

I. Abstract

This paper proposes a distributed Memory Built-In-Self Test (MBIST) architecture employing a hybrid technique for testing heterogeneous memory cores interconnected using NoC. In the proposed architecture, the memory cores are placed in different groups based on distance and timing constraints. Each group has a dedicated BIST controller which performs parallel March test on all the cores in a group while the groups are tested in a pipeline. The paper also proposes a test schedule for the proposed architecture to keep the test power within the power budget. Experiments performed on ITC'02 benchmark circuit confirms that our proposed test schedule performs a more power constrained test as compared to dedicated BIST technique. Moreover, experimental results indicate real estate benefits for the proposed distributed BIST architecture in comparison to other reported techniques.

II. Introduction

Intellectual Property (IP) based design and advancement in manufacturing technology have allowed today's Systems-On-Chip (SoCs) to include hundreds of embedded cores. In such dense SoCs, the choice of interconnect architecture is a big concern as it governs the performance, power dissipation and cost of the system. Designers using bus based interconnect network for complex chip designs often face difficulty related to bandwidth, signal integrity, and power dissipation of the chip.

A new communication architecture called Network-on-Chip (NoC) [1] was proposed to solve these issues. In a NoC-based chip, the communication network consists of network interfaces (NI), routers, and channels to connect the routers. The cores communicate among each other by sending and receiving packets.

Like all other SoCs, NoC based SoCs must also be tested for manufacturing defects. An important test issue for such NoC based SoCs is the design of an efficient Test Access Mechanism (TAM). In NoC based SoCs, the area overhead due to the communication infrastructure is large. In such a situation, a dedicated TAM solely for the purpose of test is impractical as it would further increase the area overhead.

Thus, the re-use of the NoC as TAM is a very attractive solution as it overcomes the problem of additional TAM area overhead. The reuse method assumes that most or all the cores embedded into the system are connected or accessible through the communication platform. This easier access to the cores and the high parallelism of communications provided by the network make this platform a cost-effective TAM.

Ever since, it was shown in [2] and [3] that reduced test times can be achieved by the network reuse even under power constraints while pin count and area overhead are strongly minimized, the NoC re-use for TAM has gained immense popularity among different research groups. Based on the re-use of NoC for data communication as TAM, different reseachers have proposed NoC based SoC test scheduling techniques.

A. *Motivation*

Majority of research related to re-use of NoC to act as TAM have focussed on testing of logic cores. However, not much has been done on exploring the NoC to act as TAM while testing memory cores interconnected using NoC. One probable reason may be that Built-In-Self Test (BIST) is the most preferred technique for testing memories and Memory BIST (MBIST) does not need any TAM as tests generation as well as comparison of results are done on-chip.

However, MBIST for memories connected using NoC also face the same test challenges as faced by any other BIST technique for embedded memories. Unless carefully designed, NoC based MBIST may induce excessive power, in addition to performance and area overhead. For instance, with hundreds of embedded memories, dedicated BIST for each memory core leads to high routing and gate area overhead.

To reduce the BIST area and routing overhead, distributed approaches are necessary. However, as hardware resource sharing is introduced in distributed memory BIST, the testing technique must be carefully considered to reduce the routing congestion and to facilitate rapid power-constrained testing. Parallel testing can reduce test time, but power consumption may be a factor. Sequential testing has the opposite effect. A balance of both techniques may be best. BIST signal routing also must be considered when deciding how to share

`/13 $26.00 © 2013 IEEE
DOI 10.1109/DSD.2013.67

controllers.

Although most of the above objectives have been tackled separately by previous approaches involving BIST design optimization for memory cores in SoCs, there are no system-level solutions for effective power constrained testing of hundreds of embedded memories connected using NoC (achieve high test concurrency with low overhead in Design For Testability hardware).

B. *Contribution*

1) In this work, we have proposed a distributed MBIST architecture for testing heterogeneous memory cores interconnected using NoC. The proposed architecture is an improvement on the architecture proposed in [4] as it allows test of memory cores of any size while the one proposed in [4] allowed only homogeneous memory cores. Utilizing a distributed BIST architecture leads to much lesser area overhead than dedicated BIST for each core.

   In the proposed architecture, the memory cores form different groups based on distance and timing constraints. Each group has a dedicated BIST controller which performs parallel March test on all the cores in a group. The groups are tested in a pipeline fashion. The hybrid test technique and the distributed BIST achitecture allows the test of memory cores to be performed at much lesser time than required in [5] .
2) Based on the proposed NoC based MBIST architecture, we have proposed a test schedule to optimize test time as well as keep the test power within the power budget. Experiments performed on d695 ITC'02 benchmark circuit confirms that our proposed test schedule performs a more power constrained test as compared to dedicated BIST technique.

III. Previous Work

To the best of our knowledge, NoC based MBIST has only been proposed in [4] and [5]. Liu etal. in [5] have proposed a packet based BIST scheme which tests memories in a NoC. The BIST uses the network resource to transport test patterns such that one BIST circuit can test all memories attached to the network resource without incurring the routing and timing problems.

However, the proposed work of Liu etal. has several problems. It proposes a single BIST controller for testing a number of identical memories. If we assume all memory cores are non-BISTed, then a single BIST controller has to send address, data and control for all the memories. This will substantially increase the network transport latency and hence negatively impact the test time.

However, If we assume all memories are BISTed (each memory has an Address generator, Data generator and Test pattern generator), then the area overhead problem is the same as the case where each core has a dedicated BIST. No additional advantage is gained by sharing the BIST controller.

The experimental results presented by the authors are based on the assumption that the memory cores connected to the NoC are homogeneous. Such an approximation is too simple given the complexity of today's memory intensive NoC based applications and the different sizes of memories used in them. Moreover, the authors have not experimented on any benchmark circuit to give an estimate of test time or power.

In [4], the authors have proposed the test architecture considering homogeneous memory cores and have used Particle Swarm Optimization (PSO) based technique to position the BIST controllers. The authors have considered only network transport latency as the test cost to show the effectiveness of their approach which is again a loose approximation as the test time is pre-dominantly governed by the sizes of the memory cores. Moreover, no area overhead or power dissipation estimation of the test architecture was presented.

IV. Proposed Method: NoC based MBIST

For BIST design optimization, researchers have either proposed a serial testing approach or a parallel testing approach for memory cores embedded in SoCs. Testing the memory cores one by one keeps the power dissipation under check but increases the test time while parallely testing a number of cores makes testing faster at the cost of high power dissipation [6], [7], [8],[9]. So the best solution is to leverage on the advantages of both the approaches.

We have thus proposed a hybrid test approach where both the serial and parallel testing techniques can be utilized to optimize test time as well as test power. The proposed test architecture is a distributed Memory BIST (MBIST) architecture having a number of BIST controllers each of which is responsible for testing a group of heterogeneous memory cores. The groups are tested in a pipeline while memories in a group are tested parallely. The packet based BIST re-uses the NoC to act as TAM bringing down the routing cost. Moreover, utilizing the already existing on-chip network for test purpose avoids requirement of separate test circuits and thus reduces area overhead while performing at-speed testing.

The Figures 1 and 2 give a basic idea of the hybrid approach of testing used in the proposed architecture. The NoC considered in both Figures 1 and 2 is 3x3 mesh type NoC with X-Y routing and wormhole switching. The memory cores to be tested have been grouped to form two clusters. The grouping is done based on distance relation discussed in the test scheduling problem section. After the grouping has been done, based on the timing precedence relation, the test schedule determines the order in which the BIST controllers will receive the March tests. Let the BIST controllers for the Clusters 1 and 2 be B1 and B2 respectively and the maximum test time of Cluster 1

is greater than maximum test time of Cluster 2. Hence B1 recieves March tests earlier than B2. Assume that the memory cores are tested using a March test with two March elements M1 and M2. The sequence of test operations will be as follows:

First test session: The ATE transports M1 to B1

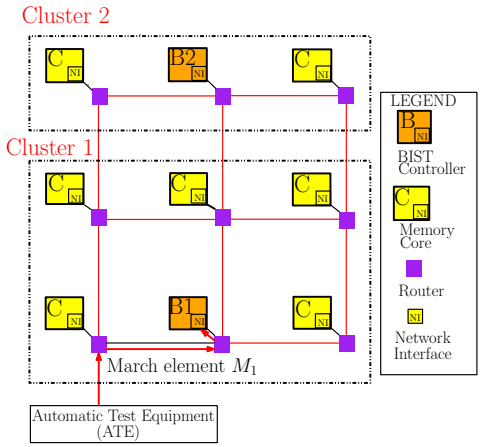


Fig. 1: Proposed NoC based MBIST Test Architecture (first test session)

(the BIST controller of the group having the core that takes the maximum test time among all cores) as shown in Figure 1. B1 decodes the instruction and accordingly generates low level memory control signals for all the memory cores in cluster 1. All the cores in Cluster 1 are tested parallely for M1. Once every core in cluster 1 passes the test for M1, the information of test completion is returned to B1.

Second test session: Once Cluster 1 completes the test for M1, B1 passes M1 to B2 and itself fetches M2 from ATE which is shown in Figure 2. B1 decodes M2, B2 decodes M1 and both B1 and B2 generate low level memory control signals for their respective clusters.

The testing time of a memory core is determined by the address space of the memory core and the length of the March algorithm. The length of the March element can be calculated by the word width of the core. Therefore, the testing time of the memory core can be computed. We assume, product of address space and word-width as a measure of testing time of a memory core. If N is the address space of a memory core and w be the word width then, the test time of the core(t) = N*w. For a group of memories tested parallely using the same March test, the testing time of an i-th core (i = 1,2.....,n) $t_i = N_i * w_i$ and the test time of the group is the maximum of all the test time of all the cores in the group.

Pipelining brings in test parallelism among groups and performs test faster as compared to other

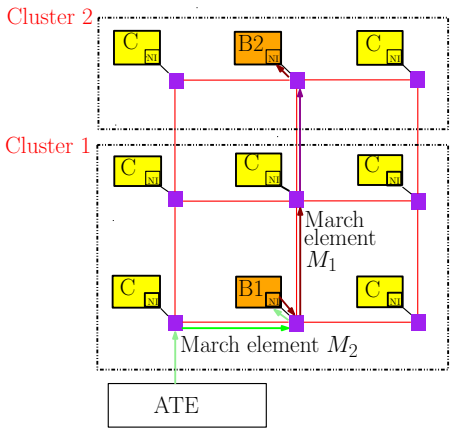


Fig. 2: Proposed NoC based MBIST Test Architecture (second test session)

proposed serial test techniques. However, as in the case of any pipelined architecture, it must be ensured that there are no pipeline stalls. Pipeline stalling in the proposed architecture can be avoided if a timing relation is maintained among the testing groups.

The group with the largest test time has to be tested first followed by the one wih the second largest and so on. For example, according to Figure 2, to avoid pipeline stalling in the second test session, Cluster 2 must finish test for M1 before Cluster 1 finishes test for M2. This precedence constraint of testing the memory core groups has to be taken care of while scheduling test for the architecture. A test schedule for the architecture based on the above timing constraint is proposed which is discussed next.

V. Test Scheduling Problem

The proposed test architecture uses a hybrid approach where number of cores in a group share a BIST controller. Thus, an obvious question that comes up is what should be the condition for group formation and how many groups should be formed? It is expected that the number of groups that are formed is minimum so that the area overhead due to the BIST controllers is minimum. Moreover, the groupings should also ensure that the power dissipation during test is within the power budget. We have thus proposed a test schedule which involves a grouping technique whose aim is to group memory cores which are at same distance from a BIST controller. Then, groupings are improved to make the test schedule satisfy the power constraint.

The proposed test scheduling strategy consists of two phases: The initial phase of memory grouping followed by the power aware test schedule phase. As shown in Figure 3, the memory grouping phase is based on the timing precedence constraint as well as on the distance constraint. Before we formally define the memory grouping problem and the power aware test

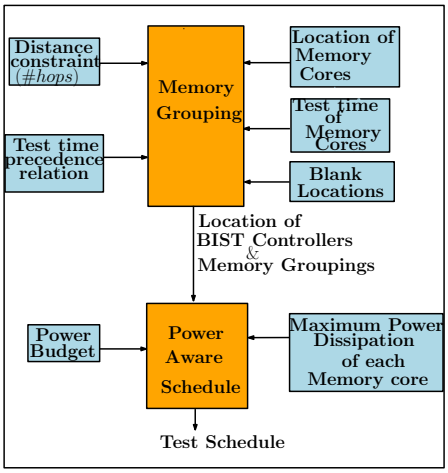


Fig. 3: Phases in the proposed test schedule

schedule, we define certain notations as follows:

n : Total number of memory cores
r : Number of rows in the NoC ; cl : Number of columns in the NoC
A : l-element march test $\{M_1,M_2....,M_l\}$ where M_l is the l-th March element
C : Set of memory cores $\{C_1,C_2...,C_n\}$
C_i : i-th core in the set C, where {i = 1,2,.....n}
$t_{C_i}(M_l)$: test time of the core C_i for the l-th March element M_l
B : Set of possible locations for placing BIST controllers $\{B_1,B_2,...,B_m\}$ where $m < n$
B_j : j-th core in the set C, where {j = 1,2,.....m}
x_{C_i}/ x_{B_j} : row number of C_i / B_j in a mesh type NoC, where {x = 1,2.....r}
y_{C_i} / y_{B_j} : column number of C_i / B_j in a mesh type NoC, where {y = 1,2,.....cl}
G : Set consisting of groups of memory cores
g_j : a memory group, $g_j \in G$ and $g_j \subseteq C$
$T_{g_j}(M_l)$: Maximum test time of a group $g_j \in G$ for March element M_l
$T_{g_j}(M_l) = Max(\forall_{C_i \in g_j} t_{C_i}(M_l))$
h : number of allowed hops in the NoC

A. *Memory Grouping Problem*

The Memory grouping problem is stated as:

Given a NoC with n number of memory cores, their testing time and their positions (in terms of row numbers and column numbers), m number of possible BIST locations and their positions, partition n into groups and allot a BIST controller to each group based on timing precedence constraint and distance constraint.

Distance constraint: The distance between the memory core and its alloted BIST controller is less than a pre-determined number of hops.

Timing precedence constraint: It requires that the group that is tested for a March element finishes the test before it receives the next March element from a group that was operating on it.

B. *The Memory Grouping Algorithm*

The objective of the memory grouping algorithm is to form clusters of memory cores where each cluster can contain memories of different sizes and the clusters be formed in such a way that the cores which are at same distance from a controller fall in one group. The groups are then ordered to maintain timing precedence relation. The memory grouping problem also assumes available blank positions where BIST controllers can be placed. These blank positions can be at the centre or along the sides or at the corner of a NoC. For example, for a NoC of size *r*cl* (where *r* is the row number and *cl* is the column number), the different positions are identified as follows:

The position is given by an ordered pair (x_j,y_j) where *x* is the row number and *y* is the column number. Corner locations are the ones which have (x_j,y_j) such that x_j can be either 1 or *r* and y_j can be either 1 or *cl*. Similarly, the side locations are the ones which satisfy the condition that if x_j can have values either 1 or *r* then j can be anything from 2 to *cl*-1 and conversely if y_j can have values either 1 or *cl* then x_j can be anything from 2 to *r*-1. The centre locations are the ones which allow x_j and y_j to allow any value within the range 2 to *r*-1 and 2 to *cl*-1 respectively.

Using a Lemma and corollary to the lemma mentioned in the next subsections, it is shown that if BIST controllers are placed at these blank locations, then for particular number of allowed hops from a BIST controller to a core, the number of cores that can be reached from a BIST controller is maximum for the controllers placed at the centre locations followed by the ones at the side and then the ones at the corner. Based on the corollary, the grouping technique uses a greedy approach of trying to allot as many cores as possible to the locations at the centre assuming that BIST controllers are placed at these locations. It is ensured that each core is alloted to exactly one controller.

After allotment of cores to all centre positons, if it is found that some memory cores remain unalloted, then the grouping algorithm tries to allot these unalloated cores to the controllers at the side locations followed by controllers at corners. After trying to allot each core to a controller in either of the three positions, still unalloted memory cores remain, then the steps for grouping are repeated with increased allowed hops.

Once every memory core has been alloted to a controller, the groups formed are the ones which share the same BIST controller. The groups are then sorted in decreasing order of test time (the test time of each group is given by the test time of the core in the group that has

maximum test time). The results of the memory grouping phase is number of memory groups (number of required BIST controllers), position of the BIST controllers and allotment of memory cores to the BIST controllers. The test architecture allows for parallel testing of memory cores in a group. Moreover, the pipelined technique of overlapped test operation of two or more groups will cause a number of memories of diferent groups to perform the same test operation. This may result in power dissipation which may exceed the power budget. Thus, a power aware schedule is required after the initial memory grouping. Based on the results of the memory grouping phase and on the power constraint, the next phase which is the improvement phase generates a power aware test schedule.

Lemma V.1. *In the proposed test architecture, for a given number of hops (h), the number of memory cores that can be reached from a blank location is given by* T_h *= (h/2) [d (1+h) + (1-h)] where d = 2 for blank locations at centre positions, d = 3 for blank locations at side positions and d = 4 for blanks at centre positions.*

Proof by induction*:* The proof is provided for only the corner case. The same approach can be used for other cases, and hence is not detailed here.

Let P(h) be the proposition that T_h=(h/2)[2(1+h) + (1-h)] where d = 2 (It means a BIST controller placed at a corner location can hop only in two directions).

Basis step : To prove P(1) is true, we assume the left most top corner. The cores that can be reached from there at one hop are cores to the right and at the bottom of the corner position only. The same logic can be applied for the other three corner positions and in each case the number of cores that can be reached is two that is T_1= 2 and (1/2)[2(1+1) + (1-1)] = 2. Hence, P(1) is true.

Inductive Step: Assume P(h) is true. Thus, T_h=(h/2) [2(1+h) + (1-h)] where d = 2 is true. It must be shown that T_{h+1} = (h+1/2)[2(1+(h+1)) + (1-(h+1))] = (h+1)(h+4)/2 where d = 2 is also true.
It can be seen that if a corner blank location is considered, then to obtain the number of cores that it reach in two hops is equal to number of cores that it can hop to in single hop plus an additional 3 places.

Extending that to case of h and h+1, we can derive that the number of cores that can be reached from a blank location at a corner position in h+1 hops is equal to places that can be reached in h hops plus an additional h+2 locations. Mathematically, it can be written as $T_{(h+1)} = T_h +$ h+2

Since we have assumed P(h) is true, then substituting the value of T_h we obtain $T_{(h+1)}$ = (h/2)[2(1+h) + (1-h)] + h+2, where d = 2
Therefore, $T_{(h+1)}$ = (h+1)(h+4) / 2. Thus, P(h+1) is true and thus proves the lemma. ■

Algorithm 1 Memory Grouping Algorithm

Input :
Size of the NoC = r*cl
Set of memory cores C and (x_{C_i},y_{C_i}) of each core $C_i \in$C
Set of possible locations for placing BIST controllers B and (x_{B_j},y_{B_j}) of each location B_j
x_{C_i}/ x_{B_j} where {x = 1,2.....r}
y_{C_i} / y_{B_j} where {y = 1,2,.....cl}
t_{C_i} : test time of the core C_i, t_{C_i} = N_i*w_i where N_i is the address space of the core C_i and w_i is the word width
p_{C_i} : maximum power dissipation of each core C_i
/* In the algorithm we assume that t_{C_i} = $t_{C_i}(M_l)$ */
Step 1 : Initialization
(a) Let the number of hops be 1.
(b) For each memory core $C_i \in$ C,∀i(1,n) use a flag (say visited(C_i)) to keep track of whether the core has been grouped (1 for grouped and 0 for ungrouped). Initialize the visited flag for each core as 0.
(c) For each blank location B_j, ∀j(1,m) create an empty set g_j. Each g_j will have a T_{g_j} representing maximum test time of the group. Initialize each T_{g_j} equal to 0.
(d) Let G be a set of g_j,∀j(1,m)

Step 2 : Sort the set of blank locations B such that the blanks at the centre positions of the NoC appear first in the sorted set followed by the blanks at the side positions and then the blanks at the corner positions. For two or more blanks at the same position, sorting is done based on increasing order of row number (x_{B_j}) followed by increasing order of column number (y_{B_j}).
/* Sorting condition:
Centre : (x_{B_j} = 2,....r-1) ∧ (y_{B_j} = 2,.....cl-1)
Side : {x_{B_j} = (1∨r)∧ y_{B_j} = (2,.....cl-1)}∨ {(x_{B_j} = (2,....r-1) ∧ (y_{B_j} = (1∨cl-1)) }
Corner :{x_{B_j} = (1∨r)} ∧ { y_{B_j} =(1∨cl))} */

Step 3 : Satisfying the distance constraint
(a)Assume a BIST controller is placed at B_j. Initialize the value of j to 1.
(b) Check whether set C = ∅. If yes, then goto Step 5
(c) For each core $C_i \in$ C,
if (dist(C_i,B_j) ≤ h) and visited(C_i)= 0 **then**
(i) Include C_i in the set g_j : g_j = $g_j \cup C_i$
(ii) Update G with new g_j : G = G ∪ g_j
(iii) Mark the core C_i as visited : visited(C_i)= 1
(iv) Check whether the test time of the core t_{C_i} is greater than the maximum test time of the group T_{g_j}. If yes, then replace T_{g_j} with t_{C_i}. This ensures that the test time of the group is same as the test time of the core which has maximum test time in the group.
(v) Drop the core C_i from the set C : C = C\ {C_i}
end if
(d) Repeat steps (b) and (c) for the rest of the blank locations $B_j \in B$,∀j(2,m)

Step 4: Grouping the memory cores which could not be grouped for h=1
while (C ≠ ∅) **do**
Repeat Step 3 with h incremented by 1
end while
Step 5 : Satisfying the timing precedence relation and the resulting grouping result
(a) Drop empty groups from G
(b) Sort G in decreasing order of T_{g_j}
(c) return (G)

Corollary V.2. *: For a particular number of allowed hops, the number of cores that can be reached from a blank location depends on the position of the blank location (the value of* d *in T_h). Greater the value of* d *more the number of cores that can be reached. Thus, a blank at the centre position of the NoC is the best place to put a BIST controller in the NoC.*

VI. Experimental results

A. *Experimental Setup*

ITC'02 benchmarks [10] were used to evaluate the proposed test architecture and the test scheduling algorithm. As the original benchmarks do not include test data for memory cores, we have added the data ourselves. We assume all cores in the benchmark are memory cores similar to [11] and use the same data for d695 as used in [11] and [12]. The test data used for d695 circuit is shown in Table I.

TABLE I: Data for d695 benchmark circuit

Core	time (units)	power (units)
1	38	660
2	1029	602
3	2507	823
4	5829	275
5	12192	690
6	11978	354
7	4219	530
8	4605	753
9	1659	641
10	7586	1144

The first step of our experiment was mapping the d695 benchmark to a NoC. We followed the same mapping as done in [13]. In [13], the authors have implemented System d695 in grid SOCIN NoC. The implementation involved mapping the 10 cores of d695 to 4x3 size mesh type NoC. Thus, two of the 12 routers are not connected to any core. We have modified the mapping by connecting two BIST controllers to the unconnected routers and each of the ten cores is assumed to be a memory core. The modified implementation of the d695 bechmark is shown in Figure 4. The proposed test scheduling algorithm was written in C and was run on a PC with 2.0 GHz processor and 2GB RAM. The input to the algorithm was the test time and test power values of d695 circuit given in Table I and the locations of the cores and BIST controllers. In the first experiment we assumed dedicated BIST for each core. Thus there was no requirement of the BIST controllers and the mapping was same as in [13]. However, we used another nine different mappings and for each mapping we calculated the total power dissipation for a (*rw*) March operation. In the next experiment, we applied our test scheduling algorithm to the modified d695 implementation as shown in Figure 4 and repeated it for another 9 different mappings.

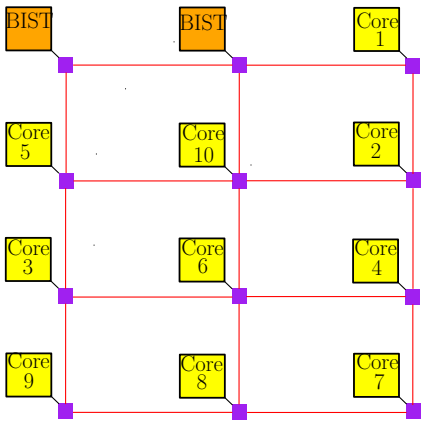


Fig. 4: Modified mapping of the cores of System d695 to 4x3 size mesh type NoC assuming all memory cores

B. *Results for the d695 benchmark circuit*

*1) **Area Estimate**:* The BIST circuit was coded in Verilog and synthesized using Faraday 180nm library. The area estimate of the individual components are given in Table II. Table III compares the area overhead of our proposed approach compared to the other approaches. The table III shows that our proposed test architecture is far better than the dedicated (parallel) BIST approach and also better than one proposed by Liu where they assumed BISTed cores. However, the proposed approach falls behind Liu's technique when non-BISTed cores are assumed. The reason is Liu's approach requires one controller whereas our proposed approach requires two controllers in the case of d695 circuit mapping.

TABLE II: Area estimate of the BIST controller synthesized in 180nm library

Component	Area (μm^2)
Controller	95
Test Pattern Generator	607

TABLE III: Comparison of the BIST area overhead

BIST Technique	Area overhead (μm^2)
Dedicated BIST	7030
Li etal.[5] (BISTed cores)	6070
Li etal.[5](non-BISTed cores)	703
Proposed	1406

*2) **Power Variation**:* Figure 5 gives the graph for the parallel BIST scheme (each core with dedicated BIST) for 10 different mappings of the cores of d695 to 4x3 mesh type NoC during the (*rw*) March operation. During the (*rw*) operation, simultaneous read is performed on all the 10 cores followed by simultaneous write on all 10 cores. Figure 5 shows that during simultaneous reads ('*r*' operation) on all 10 cores the total

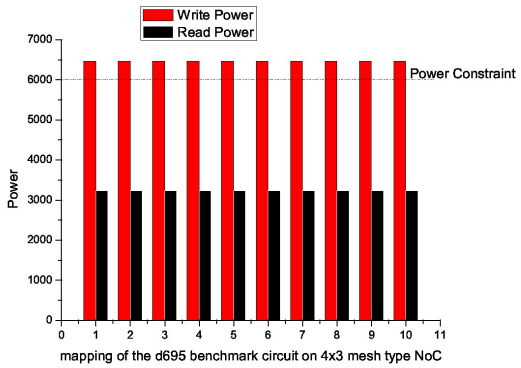


Fig. 5: Test Power variation during (*rw*) March operation on ITC'02 benchmark circuit. The cores in the benchmark circuit are mapped on 4x3 size NoC and all cores are tested parallely

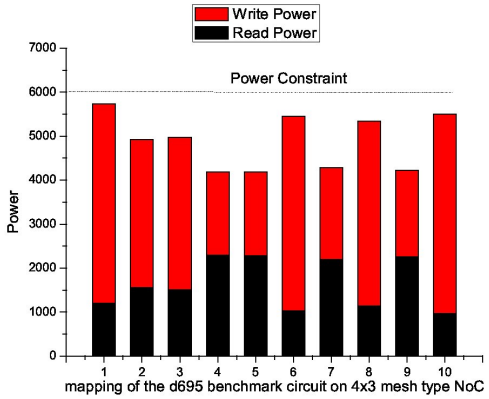


Fig. 6: Test Power variation during (rw) March operation on ITC'02 benchmark circuit. The cores in the benchmark circuit are mapped on 4x3 size NoC and all cores are tested using the proposed hybrid test approach

read power remains within the power budget. However, during the ('*w*') operation the total write power for all the cores exceeds the power constraint.

Figure 6 shows the power variation for the proposed scheduling technique with two BIST controllers for 10 different mappings of the cores to the NoC. In the graph, it shows that for each mapping of the cores the total power dissipation for (*rw*) operation is within the power constraint. The justification for such power constrained test results is explained using the mapping of d695 circuit to a 4x3 mesh type NoC as shown in Figure 4. According to the proposed test scheduling algorithm, for the mapping as shown in Figure 4, two groups of memories are formed. Memory cores 1,3,5,9 form one group and the rest the other group. According to the data in Table I, core 5 has the highest test time, and thus BIST controller of the group containing core 5 is the one which receives the March element level instructions first. Thus, during the (*rw*) operation, when the group with core 5 performs read operation, the other group performs write operation. Thus at any instant of time, the total power dissipation will be :

$$P_d = \sum_{i=3,5,9} P_r(i) + \sum_{j=1,2,4,6,7,8,10} P_w(j)$$

where P_d : total power dissipation
$P_r(i)$: read power for core *i* and
$P_w(j)$: write power for core *j*
We assume that the power values of the cores given in Table I are the power values during write operation to the core and that read power for each core is approximately half of write power [14]. Thus, power dissipation P_d, calculated using the read and write power values remains within power budget. As shown in Figure 6, for the other mappings also P_d remains below power constraint.

*3) **Test time estimate**:* We provide a time estimate of test for the mapping of Figure 4 and use the data of Table I. We assume a March test with two March elements. According to the proposed test scheduling algorithm, the Memory cores 1,3,5,9 form one group. We call it group 1. The rest of the cores form the other group. We call it group 2. The test time of group1 is given by core 5 (core having the largest test time in the group) while for group 2 the test time is the same as test time for core 6. Thus, the total test time is calculated as T = T_{g1} + T_{g2} = 12192 + 11978 = 24170. For the parallel BIST scheme, as all cores are tested parallely, the total test time is given by the core with largest test time. For the mapping of Figure 4, test time for parallel BIST scheme is same as test time of core 5 and is equal to 11292. Thus, compared to parallel BIST scheme, our proposed test technique requires additional test time equal to the test time of group 2.

VII. Conclusion

We have proposed a BIST based test architecture for memories interconnected using NoC and have also proposed a test schedule for the architecture. We experimented on ITC'02 benchmark circuit where we showed that the BIST area overhead for the proposed architecture is much less compared to other reported techniques. Experiments on the benchmark circuit for the propsoed test schedule showed that our proposed test scedule performs a more power constrained test than the dedicated BIST technique. However, the test schedule needs to be tested for other March elements consisting of more than one read write elements. Our on-going work aims to improve the proposed test schedule to make it more power aware such that it performs power constrained test for all March elements and on large number of memory groups.

References

[1] L. Benini and G. D. Micheli, "Chapter 1 - Networks on Chip," in *Networks on Chips*, pp. 1 – 22, San Francisco: Morgan

Kaufmann, 2006.

[2] E. Cota, M. Kreutz, C. Zeferino, L. Carro, M. Lubaszewski, and A. Susin, "The Impact of NoC Reuse on the Testing of Core-Based Systems," in *Proceedings of the 21st VLSI Test Symposium, 2003*, pp. 128–133, April-May 2003.

[3] E. Cota, L. Carro, F. Wagner, and M. Lubaszewski, "Power-Aware NoC Reuse on The Testing of Core-Based Systems," in *Proceedings of the International Test Conference, 2003*, vol. 1, pp. 612–621, October, 2003.

[4] B. Ghoshal, S. Kundu, I. Sengupta, and S. Chattopadhyay, "Particle Swarm Optimization Based BIST Design for Memory Cores in Mesh Based Network-on-Chip," in *Proceedings of the 16th International Symposium on VLSI Design and Test (VDAT)*, pp. 343–349, 2012.

[5] H.-N. Liu, Y.-J. Huang, and J.-F. Li, "Memory Built-in Self Test in Multicore Chips with Mesh-Based Networks," *IEEE Micro*, vol. 29, pp. 46–55, Sept.-Oct. 2009.

[6] B. Wang and Q. Xu, "Test/Repair Area Overhead Reduction for Small Embedded SRAMs," in *proceedings of the 15th Asian Test Symposium, 2006. ATS '06*, pp. 37–44, November 2006.

[7] B. Nadeau-Dostie, A. Silburt, and V. Agarwal, "A Serial Interfacing Technique for Built-in and External Testing of Embedded Memories," in *Proceedings of the IEEE Custom Integrated Circuits Conference, 1989,*, pp. 22.2/1 –22.2/5, May 1989.

[8] Y.-J. Huang and J.-F. Li, "A Low-Cost Pipelined BIST Scheme for Homogeneous RAMs in Multicore Chips," in *Proceedings of the 17th Asian Test Symposium, 2008. ATS '08*, pp. 357–362, November 2008.

[9] L.-M. Denq and C.-W. Wu, "A Hybrid BIST Scheme for Multiple Heterogeneous Embedded Memories," in *Proceedings of the 16th Asian Test Symposium, 2007. ATS '07*, pp. 349 –354, oct. 2007.

[10] E. Marinissen, V. Iyengar, and K. Chakrabarty, "A Set of Benchmarks for Modular Testing of SoCs," in *Proceedings of the International Test Conference, 2002*, pp. 519 – 528, 2002.

[11] T. Yoneda, Y. Fukuda, and H. Fujiwara, "Test Scheduling for Memory Cores with Built-In Self-Repair," in *Proceedings of the 16th Asian Test Symposium, 2007. ATS '07*, pp. 199 –206, oct. 2007.

[12] J. Pouget, E. Larsson, and Z. Peng, "Multiple-Constraint Driven System-on-Chip Test Time Optimization," *Journal of Electronic Testing: Theory and Applications*, vol. 21, no. 6, pp. 599–611, December 2005.

[13] E. Cota and C. Liu, "Constraint-Driven Test Scheduling for NoC-Based Systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 25, no. 11, pp. 2465–2478, November 2006.

[14] M. Gottscho, A. Kagalwalla, and P. Gupta, "Power Variability in Contemporary DRAMs," *Embedded Systems Letters, IEEE*, vol. 4, no. 2, pp. 37–40, 2012.